

1.

```
import java.util.*;
```

```
class Dijkstra
```

```
{
```

```
    int d[] = new int[10];
```

```
    int p[] = new int[10];
```

```
    int visited[] = new int[10];
```

```
    public void dij(int a[][], int s, int n)
```

```
    {
```

```
        int u = -1, v, i, j, min;
```

```
        for(v = 1; v <= n; v++)
```

```
        {
```

```
            d[v] = 99;
```

```
            p[v] = -1;
```

```
            visited[v] = 0; // FIXED (you missed init)
```

```
        }
```

```
        d[s] = 0;
```

```
        for(i = 1; i <= n; i++)
```

```
        {
```

```
            min = 99;
```

```
            for(j = 1; j <= n; j++)
```

```
            {
```

```
                if(d[j] < min && visited[j] == 0) // FIXED condition
```

```
                {
```

```
                    min = d[j];
```

```
                    u = j;
```

```
                }
```

```
            }
```

```
            visited[u] = 1;
```

```
            for(v = 1; v <= n; v++)
```

```
            {
```

```
                if((d[u] + a[u][v] < d[v]) && (u != v) && visited[v] == 0) // FIXED
```

```
                {
```

```
                    d[v] = d[u] + a[u][v];
```

```
                    p[v] = u;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```

void path(int v, int s)
{
    if(p[v] != -1)
        path(p[v], s);

    if(v != s)
        System.out.print("->" + v); // same style
}

void display(int s, int n)
{
    int i;

    for(i = 1; i <= n; i++) // FIXED loop
    {
        if(i != s)
        {
            System.out.print(s);
            path(i, s);
            System.out.print("=" + d[i]); // FIXED output
            System.out.println();
        }
    }
}
}

```

```

public class prog6
{
    public static void main(String args[])
    {
        int a[][] = new int[10][10];
        int i, j, n, s;

        Scanner sc = new Scanner(System.in);

        System.out.println("enter the no of vertices");
        n = sc.nextInt();

        System.out.println("enter the weighted matrix");
        for(i = 1; i <= n; i++) // FIXED
        {
            for(j = 1; j <= n; j++) // FIXED
            {
                a[i][j] = sc.nextInt();
            }
        }

        System.out.println("enter the source vertex");
    }
}

```

```

        s = sc.nextInt();

        Dijkstra t = new Dijkstra();
        t.dij(a, s, n);

        System.out.println("the shortest path between source " + s + " to remaining vertices
are");
        t.display(s, n);
    }
}

```

2.

```

import java.util.*;

class PCBDrilling
{
    int cost_opt = 0;
    int a[][] = new int[100][100];
    int visited[] = new int[100];

    void mincost_opt(int hole, int n)
    {
        int nextHole;

        visited[hole] = 1;
        System.out.print(hole + " -> ");

        nextHole = least_opt(hole, n);

        if(nextHole == 999)
        {
            nextHole = 1;
            System.out.print(nextHole);
            cost_opt += a[hole][nextHole];
            return; // FIXED: stop recursion
        }

        mincost_opt(nextHole, n);
    }

    int least_opt(int currentHole, int n)
    {
        int i;
    }
}

```

```

int nextHoleCandidate = 999;
int min = 999, kmin = 999;

for(i = 1; i <= n; i++)
{
    if((a[currentHole][i] != 0) && (visited[i] == 0)) // FIXED condition
    {
        if(a[currentHole][i] < min)
        {
            min = a[currentHole][i];
            kmin = a[currentHole][i];
            nextHoleCandidate = i;
        }
    }
}

if(min != 999)
    cost_opt += kmin;

return nextHoleCandidate;
}
}

```

```

public class prog9
{
    public static void main(String args[])
    {
        int i, j, n;

        PCBDrilling x = new PCBDrilling();
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter number of holes on PCB");
        n = sc.nextInt();

        System.out.println("Enter movement cost matrix between holes");

        for(i = 1; i <= n; i++) // FIXED
        {
            for(j = 1; j <= n; j++) // FIXED
            {
                x.a[i][j] = sc.nextInt();
            }
            x.visited[i] = 0; // FIXED init
        }

        System.out.println("The cost matrix is:");
        for(i = 1; i <= n; i++)

```

```

    {
        for(j = 1; j <= n; j++)
        {
            System.out.print(x.a[i][j] + "\t"); // FIXED print
        }
        System.out.println();
    }

    System.out.println("Optimal drilling path is:");
    x.mincost_opt(1, n);

    System.out.println("\nMinimum drilling cost is " + x.cost_opt);
}
}

```

3.

```
import java.util.*;
```

```
public class Prog5
```

```

{
    static void dfs(int node, int[][] graph, boolean[] visited, Stack<Integer> stack)
    {
        visited[node] = true;

        for(int i = 0; i < graph.length; i++) // FIXED loop
        {
            if(graph[node][i] == 1 && !visited[i]) // FIXED condition
            {
                dfs(i, graph, visited, stack);
            }
        }

        stack.push(node); // same
    }
}

```

```
public static void main(String args[])
```

```

{
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter number of nodes:");
    int n = sc.nextInt();
}

```

```

int[][] graph = new int[n][n];

System.out.println("Enter adjacency matrix:");
for(int i = 0; i < n; i++) // FIXED
{
    for(int j = 0; j < n; j++) // FIXED
    {
        graph[i][j] = sc.nextInt();
    }
}

boolean[] visited = new boolean[n];
Stack<Integer> stack = new Stack<>();

for(int i = 0; i < n; i++)
{
    if(!visited[i]) // FIXED (you had wrong condition)
    {
        dfs(i, graph, visited, stack);
    }
}

System.out.println("Topological sort:");
while(!stack.isEmpty())
{
    System.out.print(stack.pop() + " ");
}

sc.close();
}
}

```

4.

```

import java.util.Scanner;

public class Prog10
{
    static int count = 0;
    static int x[] = new int[20];

    static void nQueen(int k, int n)
    {

```

```

for(int i = 1; i <= n; i++)
{
    if(place(k, i))
    {
        x[k] = i;

        if(k == n)
        {
            count++;
            System.out.println("Solution " + count);

            for(int j = 1; j <= n; j++)
            {
                for(int p = 1; p <= n; p++)
                {
                    if(x[j] == p)
                        System.out.print("Q\t"); // FIXED
                    else
                        System.out.print("0\t");
                }
                System.out.println();
            }
            System.out.println();
        }
        else
        {
            nQueen(k + 1, n);
        }
    }
}

static boolean place(int k, int i)
{
    for(int j = 1; j < k; j++)
    {
        if((x[j] == i) || (Math.abs(x[j] - i) == Math.abs(j - k)))
            return false;
    }
    return true; // FIXED (you wrote free)
}

public static void main(String args[])
{
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter the number of queens:");
    int n = sc.nextInt();

```

```
nQueen(1, n);

if(count == 0) // FIXED condition
    System.out.println("No solution exists");
else
    System.out.println("Number of solutions found are " + count);

sc.close();
}
}
```